old-kyria.c

7 layers · 50 keys · 9 combos

every arrow starts on the key that triggers it

how to read

🟪 pink key	press it to go to another layer
🟪 violet key	press it to toggle a layer on and off
🟥 red key(s)	held down while you are on this layer
→ solid arrow	hold
⇢ dashed arrow	toggle / go to / one-shot
combo combo	press the red keys together

0 rsthd
base layer
Tab Z C Caps Word + Xcase P X M ' U Q |
Xcase Xcase
/ R S T H D L N A I O ;
Alt GUI Ctrl Shift Shift Ctrl GUI Alt
_ My Lsft V Caps Word -> B Leader ▽ ▽ Leader W Caps Word : . My Lsft -
Start Leading Start Leading
▽ GUI Esc Space Enter My Bspc E Tab RGUI ▽
nav sym num fun
hold
hold
hold
hold
go to
go to
go to
go to
6 fun
▽ ▽ F7 Caps Word + Xcase ▽ ▽ ▽ ▽ ▽ ▽ ▽
▽ F1 F2 F3 F10 F11 ▽ Shift Ctrl GUI Alt ▽
▽ ▽ F4 Caps Word F12 ▽ ▽ ▽ ▽ ▽ Caps Word ▽ ▽ ▽
▽ ▽ rsthd ▽ ▽ ▽ ▽ ▽ ▽
go to
3 num
▽ ▽ < Caps Word + Xcase ▽ ▽ ▽ = ▽ ▽ ▽
/ 3 2 1 0 4 5 6 7 8 9 ▽
Alt GUI Ctrl Shift Shift Ctrl GUI Alt
▽ ▽ + Caps Word ▽ ▽ ▽ ▽ ▽ ▽ Caps Word . ▽ ▽
▽ ▽ rsthd ▽ ▽ ▽ ▽ ▽ ▽
go to
4 sym
▽ ▽ ! Caps Word + Xcase ▽ ▽ ~ = \ ▽ ▽
▽ # < - > ` & { ( ) | ▽
▽ ▽ + Caps Word _ ▽ ▽ ▽ ▽ % Caps Word] ▽ ▽
▽ ▽ rsthd ▽ ▽ ▽ ▽ ▽ ▽
go to
5 nav
▽ ▽ PrtSc Caps Word + Xcase My Paste ▽ ▽ ▽ Vi U ▽ ▽
▽ End Next Play/ Pause 🔊 Del ← ↓ ↑ → Home ▽
▽ ▽ Prev Caps Word ▽ ▽ ▽ ▽ ▽ Vi B Caps Word Vi W ▽ ▽
▽ ▽ rsthd ▽ ▽ ▽ ▽ ▽ ▽
go to
1 qwerty
Tab Q W Caps Word + Xcase T Y U I O P |
My Esc A S D F G H J K L ; '
/ Z X Caps Word B Leader ▽ ▽ Leader N Caps Word . / -
▽ GUI My Lalt My Lsft My Ent My Bspc My Spc My Rctl RGUI ▽
2 gaming
Esc Z Q Caps Word + Xcase R Y U I O P |
Tab Shift A S D F H J K L ; '
Ctrl Ctrl Z Caps Word C Leader ▽ ▽ Leader N Caps Word . / -
▽ GUI Z Space Enter My Bspc My Spc My Rctl RGUI ▽

old-kyria.c

keymap source, comments removed

```c
#include QMK_KEYBOARD_H
#include <string.h>
#include "features/casemodes.h"
#include "features/leader.h"

#ifdef USE_VIM
#include "qmk-vim/src/vim.h"
#include "qmk-vim/src/modes.h"
#endif

enum layers {
    _RSTHD = 0,
    _QWERTY,
    _GAMING,
    _NUM,
    _SYM,
    _NAV,
    _FUN,
};

enum custom_keycodes {
    CAPSWRD = SAFE_RANGE,
    LEADER,
};

#define MY_HOMEROW_LAYOUT( \
    L00, L01, L02, L03, L04, L05,                     R06, R07, R08, R09, R10, R11, \
    L12, L13, L14, L15, L16, L17,                     R18, R19, R20, R21, R22, R23, \
    L24, L25, L26, L27, L28, L29, L30, L31, R32, R33, R34, R35, R36, R37, R38, R39, \
                   L40, L41, L42, L43, L44, R45, R46, R47, R48, R49 \
    ) \
    LAYOUT( \
        L00, L01, L02, L03, L04, L05,                     R06, R07, R08, R09, R10, R11, \
        L12, LALT_T(L13), LGUI_T(L14), LCTL_T(L15), LSFT_T(L16), L17,   \
        R18, LSFT_T(R19), LCTL_T(R20), LGUI_T(R21), LALT_T(R22), R23,   \
        L24, L25, L26, L27, L28, L29, L30, L31, R32, R33, R34, R35, R36, R37, R38, R39, \
                    L40, L41, L42, L43, L44, R45, R46, R47, R48, R49 \
    )

#define MY_ALPHA_LAYOUT( \
    K01, K02, K03, K04, K05, K06, K07, K08, K09, K0A, \
    K11, K12, K13, K14, K15, K16, K17, K18, K19, K1A, KQUOT, \
    K21, K22, K23, K24, K25, K26, K27, K28, K29, K2A, \
    KTHMB \
    ) \
    LAYOUT( \
      KC_TAB,   K01,   K02,   K03,     K04,     K05,                                          K06,     K07,    K08,    K09,    K0A,   KC_PIPE, \
      MY_ESC,   K11,   K12,   K13,     K14,     K15,                                          K16,     K17,    K18,    K19,    K1A,   KQUOT, \
      KC_SLSH,  K21,   K22,   K23,     K24,     K25,     LEADER,  _______, _______, LEADER,   K26,     K27,    K28,    K29,    K2A,   KC_MINS, \
                              _______, KC_LGUI, MY_LALT, KTHMB,   MY_ENT,  MY_BSPC, MY_SPC,   MY_RCTL, KC_RGUI, _______ \
    )

const uint16_t PROGMEM keymaps[][MATRIX_ROWS][MATRIX_COLS] = {
#define RST_E LT(_NUM, KC_E)
#define RST_SPC LT(_NAV, KC_SPC)
#define RST_ENT LT(_SYM, KC_ENT)
#define RST_TAB LT(_FUN, KC_TAB)
    [_RSTHD] = MY_HOMEROW_LAYOUT(
        KC_TAB,   KC_Z,    KC_C, KC_Y, KC_F, KC_P,                                     KC_X, KC_M, KC_QUOT, KC_U,    KC_Q,    KC_PIPE,
        KC_SLSH,  KC_R,    KC_S, KC_T, KC_H, KC_D,                                     KC_L, KC_N, KC_A,    KC_I,    KC_O,    KC_SCLN,
        KC_UNDS,  MY_LSFT, KC_V, KC_G, KC_K, KC_B, LEADER,  _______, _______, LEADER,  KC_W, KC_J, KC_COMM, KC_DOT,  MY_LSFT, KC_MINS,
                        _______, KC_LGUI, KC_ESC, RST_SPC, RST_ENT, MY_BSPC, RST_E, RST_TAB, KC_RGUI, _______
    ),
    [_QWERTY] = MY_ALPHA_LAYOUT(
        KC_Q,   KC_W,   KC_E,   KC_R,   KC_T,           KC_Y,   KC_U,   KC_I,    KC_O,   KC_P,
        KC_A,   KC_S,   KC_D,   KC_F,   KC_G,           KC_H,   KC_J,   KC_K,    KC_L,   KC_SCLN, KC_QUOT,
        KC_Z,   KC_X,   KC_C,   KC_V,   KC_B,           KC_N,   KC_M,   KC_COMM, KC_DOT, KC_SLSH,
        MY_LSFT
    ),
    [_GAMING] = LAYOUT(
      KC_ESC,   KC_Z,   KC_Q,   KC_W,     KC_E,     KC_R,                                             KC_Y,   KC_U,   KC_I,    KC_O,   KC_P,       KC_PIPE,
      KC_TAB,   KC_LSFT,   KC_A,   KC_S,     KC_D,     KC_F,                                          KC_H,   KC_J,   KC_K,    KC_L,   KC_SCLN,    KC_QUOT,
      KC_LCTL,  KC_LCTL,   KC_Z,   KC_X,     KC_X,     KC_C,     LEADER,  _______, _______, LEADER,   KC_N,   KC_M,   KC_COMM, KC_DOT, KC_SLSH,    KC_MINS,
                              _______, KC_LGUI, KC_Z, KC_SPC,   KC_ENT,  MY_BSPC, MY_SPC,   MY_RCTL, KC_RGUI, _______
    ),
    [_NUM] = MY_HOMEROW_LAYOUT(
      _______, _______, KC_LABK, KC_MINS, KC_RABK, _______,                                     _______, _______, KC_EQL,  _______, _______, _______,
      KC_SLSH, KC_3,    KC_2,    KC_1,    KC_0,    KC_4,                                        KC_5,    KC_6,    KC_7,    KC_8,    KC_9,    _______,
      _______, _______, KC_PLUS, KC_CIRC, KC_ASTR, _______, _______, _______, _______, _______, _______, _______, KC_COMM, KC_DOT,  _______, _______,
                                 _______, _______, TO(0),   _______, _______, _______, _______, _______, _______, _______
    ),
    [_SYM] = LAYOUT(
      _______, _______, KC_EXLM, KC_AT,   KC_DLR,  _______,                                     _______, KC_TILD, KC_EQL,  KC_BSLS, _______, _______,
      _______, KC_HASH, KC_LABK, KC_MINS, KC_RABK, KC_GRV,                                      KC_AMPR, KC_LCBR, KC_LPRN, KC_RPRN, KC_PIPE, _______,
      _______, _______, KC_PLUS, KC_CIRC, KC_ASTR, KC_UNDS, _______, _______, _______, _______, KC_PERC, KC_RCBR, KC_LBRC, KC_RBRC, _______, _______,
                                 _______, _______, TO(0),   _______, _______, _______, _______, _______, _______, _______
    ),
 #define LCSPSCR LSFT(LCTL(KC_PSCR))
    [_NAV] = LAYOUT(
      _______, _______, KC_PSCR, MY_COPY, LCSPSCR, MY_PASTE,                                    _______, _______, _______, VI_U,    _______, _______,
      _______, KC_END,  KC_MNXT, KC_MPLY, KC_VOLU, KC_DEL,                                      KC_LEFT, KC_DOWN, KC_UP,   KC_RGHT, KC_HOME, _______,
      _______, _______, KC_MPRV, KC_MUTE, KC_VOLD, _______, _______, _______, _______, _______, VI_B,    KC_PGDN, KC_PGUP, VI_W,    _______, _______,
                                 _______, _______, TO(0),   _______, _______, _______, _______, _______, _______, _______
    ),
    [_FUN] = LAYOUT(
      _______, _______, KC_F7,   KC_F8,   KC_F9,   _______,                                     _______, _______, _______, _______, _______, _______,
      _______, KC_F1,   KC_F2,   KC_F3,   KC_F10,  KC_F11,                                      _______, KC_LSFT, KC_LCTL, KC_LGUI, KC_LALT, _______,
      _______, _______, KC_F4,   KC_F5,   KC_F6,   KC_F12,  _______, _______, _______, _______, _______, _______, _______, _______, _______, _______,
                                 _______, _______, TO(0),   _______, _______, _______, _______, _______, _______, _______
    ),
};

bool use_default_xcase_separator(uint16_t keycode, const keyrecord_t *record) {
    switch (keycode) {
        case KC_A ... KC_Z:
        case KC_1 ... KC_0:
            return true;
    }
    return false;
}

bool process_record_user(uint16_t keycode, keyrecord_t *record) {
    if (!process_case_modes(keycode, record)) {
        return false;
    }

    if (!process_leader(keycode, record)) {
        return false;
    }

#ifdef USE_VIM
    if (!process_vim_mode(keycode, record)) {
        return false;
    }
#endif

    switch (keycode) {
        case CAPSWRD:
            if (record->event.pressed) {
                toggle_caps_word();
            }
            return false;
        case LEADER:
            if (record->event.pressed) {
                start_leading();
            }
            return false;
        case LCTL_T(KC_PLUS):
            if (record->event.pressed && record->tap.count > 0) {
                tap_code16(KC_PLUS);
                return false;
            }
            return true;
        case LGUI_T(KC_RABK):
            if (record->event.pressed && record->tap.count > 0) {
                tap_code16(KC_RABK);
                return false;
            }
            return true;
        default:
            return true;
    }
}

#ifdef COMBO_ENABLE
enum combo_events {
  PD_COMBO,
  XL_COMBO,
  LEFT_LEAD,
  RIGHT_LEAD,
  YF_COMBO,
  GK_COMBO,
  JCOMM_COMBO,
  THSPC_COMBO,
  JA_COMBO,
};

const uint16_t PROGMEM pd_combo[] = {KC_P, KC_D, COMBO_END};
const uint16_t PROGMEM xl_combo[] = {KC_X, KC_L, COMBO_END};
const uint16_t PROGMEM left_thumb_combo[] = {RST_E, LEADER,  COMBO_END};
const uint16_t PROGMEM right_thumb_combo[] = {RST_SPC, LEADER,  COMBO_END};
const uint16_t PROGMEM yf_combo[] = {KC_Y, KC_F,  COMBO_END};
const uint16_t PROGMEM gk_combo[] = {KC_G, KC_K,  COMBO_END};
const uint16_t PROGMEM jcomma_combo[] = {KC_J, KC_COMM,  COMBO_END};
const uint16_t PROGMEM thspc_combo[] = {LCTL_T(KC_T), LSFT_T(KC_H), RST_SPC, COMBO_END};
const uint16_t PROGMEM ja_combo[] = {KC_J, LCTL_T(KC_A), COMBO_END};

combo_t key_combos[COMBO_COUNT] = {
  [ PD_COMBO ] = COMBO_ACTION(pd_combo),
  [ XL_COMBO ] = COMBO_ACTION(xl_combo),
  [ LEFT_LEAD ] = COMBO_ACTION(left_thumb_combo),
  [ RIGHT_LEAD ] = COMBO_ACTION(right_thumb_combo),
  [ YF_COMBO ] = COMBO_ACTION(yf_combo),
  [ GK_COMBO ] = COMBO_ACTION(gk_combo),
  [ JCOMM_COMBO ] = COMBO_ACTION(jcomma_combo),
  [ THSPC_COMBO ] = COMBO_ACTION(thspc_combo),
  [ JA_COMBO ] = COMBO_ACTION(ja_combo),
};

void process_combo_event(uint16_t combo_index, bool pressed) {
    switch (combo_index) {
        case LEFT_LEAD:
        case RIGHT_LEAD:
            if (pressed) {
                start_leading();
            }
            break;
        case YF_COMBO:
            if (pressed) {
                toggle_caps_word();
                enable_xcase();
            }
            break;
        case PD_COMBO:
        case XL_COMBO:
            if (pressed) {
                enable_xcase();
            }
            break;
        case GK_COMBO:
        case JCOMM_COMBO:
            if (pressed) {
                toggle_caps_word();
            }
            break;
        case THSPC_COMBO:
            if (pressed) {
                tap_code(KC_MINS);
                tap_code16(KC_RABK);
            }
            break;
        case JA_COMBO:
            if (pressed) {
                tap_code16(KC_COLON);
            }
            break;
    }
}
#    endif

void *leader_layers_func(uint16_t keycode) {
    switch (keycode) {
        case KC_R:
            layer_move(_RSTHD);
            break;
        case KC_Q:
            layer_move(_QWERTY);
            break;
        case KC_G:
            layer_move(_GAMING);
            break;
        default:
            break;
    }
    return NULL;
}

void *leader_toggles_func(uint16_t keycode) {
    switch (keycode) {
        case KC_V:
#ifdef USE_VIM
            toggle_vim_mode();
#endif
            break;
        case KC_N:
            layer_invert(_NUM);
            break;
        default:
            break;
    }
    return NULL;
}

void *leader_start_func(uint16_t keycode) {
    switch (keycode) {
        case KC_L:
            return leader_layers_func;
        case KC_T:
            return leader_toggles_func;
        case KC_R:
            reset_keyboard();
            return NULL;
        case KC_V:
#ifdef USE_VIM
            start_oneshot_vim();
#endif
            return NULL;
        default:
            return NULL;
    }
    return NULL;
}

#ifdef USE_VIM
bool process_insert_mode_user(uint16_t keycode, const keyrecord_t *record) {
    if (record->event.pressed && keycode == LCTL(KC_S)) {
        normal_mode();
        tap_code16(keycode);
        return false;
    }
    return true;
}

bool process_visual_line_mode_user(uint16_t keycode, const keyrecord_t *record) {
    if (record->event.pressed && ((keycode & 0xFF) == KC_TAB)) {
        tap_code16(keycode);
        return false;
    }
    return true;
}
#endif

#ifdef OLED_ENABLE
oled_rotation_t oled_init_user(oled_rotation_t rotation) {
    return OLED_ROTATION_180;
}

static void render_kyria_logo(void) {
    static const char PROGMEM kyria_logo[] = {
          0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,128,128,192,224,240,112,120, 56, 60, 28, 30, 14, 14, 14,  7,  7,  7,  7,  7,  7,  7,  7,  7,  7,  7,  7,  7,  7,  7,  7, 14, 14, 14, 30, 28, 60, 56,120,112,240,224,192,128,128,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
          0,  0,  0,  0,  0,  0,  0,192,224,240,124, 62, 31, 15,  7,  3,  1,128,192,224,240,120, 56, 60, 28, 30, 14, 14,  7,  7,135,231,127, 31,255,255, 31,127,231,135,  7,  7, 14, 14, 30, 28, 60, 56,120,240,224,192,128,  1,  3,  7, 15, 31, 62,124,240,224,192,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
          0,  0,  0,  0,240,252,255, 31,  7,  1,  0,  0,192,240,252,254,255,247,243,177,176, 48, 48, 48, 48, 48, 48, 48,120,254,135,  1,  0,  0,255,255,  0,  0,  1,135,254,120, 48, 48, 48, 48, 48, 48, 48,176,177,243,247,255,254,252,240,192,  0,  0,  1,  7, 31,255,252,240,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
          0,  0,  0,255,255,255,  0,  0,  0,  0,  0,254,255,255,  1,  1,  7, 30,120,225,129,131,131,134,134,140,140,152,152,177,183,254,248,224,255,255,224,248,254,183,177,152,152,140,140,134,134,131,131,129,225,120, 30,  7,  1,  1,255,255,254,  0,  0,  0,  0,  0,255,255,255,  0,  0,  0,  0,255,255,  0,  0,192,192, 48, 48,  0,  0,240,240,  0,  0,  0,  0,  0,  0,240,240,  0,  0,240,240,192,192, 48, 48, 48, 48,192,192,  0,  0, 48, 48,243,243,  0,  0,  0,  0,  0,  0, 48, 48, 48, 48, 48, 48,192,192,  0,  0,  0,  0,  0,
          0,  0,  0,255,255,255,  0,  0,  0,  0,  0,127,255,255,128,128,224,120, 30,135,129,193,193, 97, 97, 49, 49, 25, 25,141,237,127, 31,  7,255,255,  7, 31,127,237,141, 25, 25, 49, 49, 97, 97,193,193,129,135, 30,120,224,128,128,255,255,127,  0,  0,  0,  0,  0,255,255,255,  0,  0,  0,  0, 63, 63,  3,  3, 12, 12, 48, 48,  0,  0,  0,  0, 51, 51, 51, 51, 51, 51, 15, 15,  0,  0, 63, 63,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0, 48, 48, 63, 63, 48, 48,  0,  0, 12, 12, 51, 51, 51, 51, 51, 51, 63, 63,  0,  0,  0,  0,  0,
          0,  0,  0,  0, 15, 63,255,248,224,128,  0,  0,  3, 15, 63,127,255,239,207,141, 13, 12, 12, 12, 12, 12, 12, 12, 30,127,225,128,  0,  0,255,255,  0,  0,128,225,127, 30, 12, 12, 12, 12, 12, 12, 12, 13,141,207,239,255,127, 63, 15,  3,  0,  0,128,224,248,255, 63, 15,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
          0,  0,  0,  0,  0,  0,  0,  3,  7, 15, 62,124,248,240,224,192,128,  1,  3,  7, 15, 30, 28, 60, 56,120,112,112,224,224,225,231,254,248,255,255,248,254,231,225,224,224,112,112,120, 56, 60, 28, 30, 15,  7,  3,  1,128,192,224,240,248,124, 62, 15,  7,  3,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
          0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  1,  1,  3,  7, 15, 14, 30, 28, 60, 56,120,112,112,112,224,224,224,224,224,224,224,224,224,224,224,224,224,224,224,224,112,112,112,120, 56, 60, 28, 30, 14, 15,  7,  3,  1,  1,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0
    };
    oled_write_raw_P(kyria_logo, sizeof(kyria_logo));
}

static void render_qmk_logo(void) {
  static const char PROGMEM qmk_logo[] = {
    0x80,0x81,0x82,0x83,0x84,0x85,0x86,0x87,0x88,0x89,0x8a,0x8b,0x8c,0x8d,0x8e,0x8f,0x90,0x91,0x92,0x93,0x94,
    0xa0,0xa1,0xa2,0xa3,0xa4,0xa5,0xa6,0xa7,0xa8,0xa9,0xaa,0xab,0xac,0xad,0xae,0xaf,0xb0,0xb1,0xb2,0xb3,0xb4,
    0xc0,0xc1,0xc2,0xc3,0xc4,0xc5,0xc6,0xc7,0xc8,0xc9,0xca,0xcb,0xcc,0xcd,0xce,0xcf,0xd0,0xd1,0xd2,0xd3,0xd4,0};

  oled_write_P(qmk_logo, false);
}

static void render_status(void) {
    render_qmk_logo();
    oled_write_P(PSTR("Kyria - ajrae\n\n"), false);

    oled_write_P(PSTR("Layer: "), false);
    switch (get_highest_layer(layer_state)) {
        case _RSTHD:
            oled_write_P(PSTR("RSTHD"), false);
            break;
        case _QWERTY:
            oled_write_P(PSTR("QWERTY"), false);
            break;
        case _GAMING:
            oled_write_P(PSTR("GAMING"), false);
            break;
        case _NAV:
            oled_write_P(PSTR("Navigation"), false);
            break;
        case _NUM:
            oled_write_P(PSTR("Numbers"), false);
            break;
        case _SYM:
            oled_write_P(PSTR("Symbols"), false);
            break;
        case _FUN:
            oled_write_P(PSTR("Function Keys"), false);
            break;
        default:
            oled_write_P(PSTR("Undefined"), false);
    }
    oled_write_P(PSTR("\n\n"), false);

#ifdef LEADER_DISPLAY_STR
    static uint16_t timer = 0;
    if (is_leading()) {
        oled_write_ln(leader_display_str(), false);
        timer = timer_read();
    }
    else if (vim_mode_enabled()) {
        switch (get_vim_mode()) {
            case NORMAL_MODE:
                oled_write_P(PSTR("-- NORMAL --\n"), false);
                break;
            case INSERT_MODE:
                oled_write_P(PSTR("-- INSERT --\n"), false);
                break;
            case VISUAL_MODE:
                oled_write_P(PSTR("-- VISUAL --\n"), false);
                break;
            case VISUAL_LINE_MODE:
                oled_write_P(PSTR("-- VISUAL LINE --\n"), false);
                break;
            default:
                oled_write_P(PSTR("?????\n"), false);
                break;
        }
    }
    else if (timer_elapsed(timer) < 175){
        oled_write_ln(leader_display_str(), false);
    } else {
        timer = timer_read() - 200;
        oled_write_ln("", false);
    }
#endif
}

void oled_task_user(void) {
    if (is_keyboard_master()) {
        render_status();
    } else {
        render_kyria_logo();
    }
}

#endif
```